

RAG PIPELINE

Retrieval-Augmented Generation

A Complete Step-by-Step Technical Reference

Pipeline Overview

#	Stage	One-line summary
01	Parsing	Extracting raw content from source documents
02	Chunking	Splitting text into retrievable units
03	Embedding	Turning text into semantic vectors
04	Ingesting	Storing vectors and metadata in a vector database
05	Retrieval	Finding the most relevant chunks at query time
06	Reranking	Refining candidate results before sending to the LLM

Detailed Stage Reference

STEP 01 **PARSING**

Extracting raw content from source documents

WHAT	Parsing is the first stage of a RAG pipeline. Raw source files — PDFs, Word documents, HTML pages, scanned images, or database exports — are ingested and converted into clean, machine-readable plain text (or structured Markdown).
HOW	– PDF/DOCX extraction: libraries like PyMuPDF, pdfplumber, or Apache Tika read binary formats and output text, tables, and metadata. – HTML parsing: BeautifulSoup or Trafilatura strips tags, scripts, and boilerplate, keeping only meaningful content. – OCR: Scanned documents pass through Tesseract or AWS Textract to recognize characters from pixels. – Layout preservation: advanced parsers retain headings, list structure, and table alignment to aid downstream chunking.

WHY	<i>Garbage in, garbage out. If the parser misreads a table or drops a heading, no downstream component can recover that information. Clean, structure-aware parsing is the single biggest lever on retrieval quality.</i>
TIPS	✓ Use layout-aware parsers (Unstructured.io, LlamaParse) for multi-column PDFs. ✓ Strip headers, footers, and page numbers that add noise to chunks. ✓ Preserve section titles — they provide vital context during retrieval.

STEP 02 CHUNKING

Splitting text into retrievable units

WHAT	Chunking divides the parsed text into smaller, self-contained passages. These passages become the atomic units that are embedded and later returned to the language model as context.
HOW	– Fixed-size chunking: split every N tokens (e.g. 512) with an overlap window (e.g. 50 tokens) to avoid cutting mid-sentence. – Recursive character splitting: LangChain's RecursiveCharacterTextSplitter tries paragraph → sentence → word boundaries in order. – Semantic chunking: embed sentences and cut where cosine similarity drops, keeping topically coherent blocks together. – Document-structure chunking: respect headings and sections as natural split points, keeping each section as one chunk.
WHY	<i>LLM context windows are finite. Overly large chunks waste tokens on irrelevant text; overly small chunks lose context and make retrieval noisy. The right chunk size balances precision and completeness.</i>
TIPS	✓ Aim for 256–512 tokens per chunk for most use cases; try 128 tokens for highly granular Q&A. ✓ Always include a title or breadcrumb prefix in each chunk so it is self-explanatory when retrieved in isolation. ✓ Overlap consecutive chunks by 10–15% to prevent answers from falling through the cracks at boundaries.

STEP 03 EMBEDDING

Turning text into semantic vectors

WHAT	An embedding model encodes each chunk as a dense numeric vector (typically 768–3072 dimensions) that captures its semantic meaning. Semantically similar text ends up close together in this high-dimensional space.
-------------	--

HOW	– Encoder-only transformer models (e.g. OpenAI text-embedding-3-large, Cohere embed-v3, BGE-M3) process a chunk and output a single pooled vector. – Bi-encoder architecture: query and document are embedded independently, enabling fast approximate nearest-neighbour (ANN) search at query time. – Late interaction models (ColBERT): produce per-token vectors for finer-grained matching at the cost of higher storage. – Multilingual models embed text across languages into a shared space, enabling cross-lingual retrieval.
WHY	<i>Embeddings are what make semantic search possible. Without them, retrieval would rely on brittle keyword matching (BM25) and miss synonyms, paraphrases, or conceptual relationships.</i>
TIPS	✓ Match your embedding model to your domain — a general model may underperform on legal or biomedical text. ✓ Embed both chunk text and any appended metadata (title, date) for richer representations. ✓ Normalize vectors to unit length before storage to speed up cosine similarity computation.

STEP 04 INGESTING

Storing vectors and metadata in a vector database

WHAT	Ingestion writes the computed embedding vectors — together with the original chunk text and metadata — into a persistent vector store. This index is queried at runtime to find relevant chunks for any user question.
HOW	– Vector databases (Pinecone, Weaviate, Qdrant, pgvector, Chroma) build an ANN index (HNSW, IVF-Flat, ScaNN) over stored vectors. – Each record stores: the vector, the raw chunk text, and a metadata payload (source filename, page number, date, section heading). – Batch upserts are preferred over single-record writes to maximise throughput during initial indexing. – Hybrid stores (e.g. Weaviate, Elasticsearch) co-locate BM25 sparse indices alongside dense vectors for hybrid retrieval.
WHY	<i>The vector store is the retrieval engine's memory. Its index structure determines query latency and recall. Metadata filtering (e.g. 'only search documents from 2024') dramatically narrows search space and improves precision.</i>
TIPS	✓ Store source document IDs to enable citation and provenance tracking in responses. ✓ Use namespaces or collection-level partitioning to isolate tenants or document sets. ✓ Re-index periodically when upstream documents are updated — stale chunks mislead the model.

STEP 05 RERANKING

Refining candidate results before sending to the LLM

WHAT	Reranking is a second-pass scoring step that re-scores the top-K retrieved chunks using a more powerful cross-encoder model. The cross-encoder sees both the query and each candidate chunk simultaneously, enabling much richer relevance judgements than the bi-encoder used at retrieval time.
HOW	– Cross-encoder models (BGE-Reranker, ms-marco) read the concatenation of (query, chunk) and output a scalar relevance score. – Because both inputs are processed jointly, the cross-encoder captures fine-grained query–document interaction that bi-encoders miss. – The top-N reranked chunks (e.g. top-3 or top-5) are assembled into the final LLM prompt context. – LLM-based reranking: some pipelines ask the LLM itself to rate relevance, though this is slower and costlier.
WHY	<i>Bi-encoders trade accuracy for speed. The retrieval step casts a wide net; the reranker sharpens precision before expensive LLM inference. A reranker typically boosts answer accuracy by 10–25% at low added latency.</i>
TIPS	✓ Use a domain-specific reranker if available — general rerankers may misjudge relevance in technical domains. ✓ Rerank at most 20–50 candidates; more than that starts to exceed cross-encoder latency budgets. ✓ Consider 'lost in the middle' ordering: place the most relevant chunks at the start and end of the context window, not the middle.

STEP 06 RETRIEVAL

Finding the most relevant chunks at query time

WHAT	At inference time, the user's query is embedded using the same model used during ingestion. The resulting query vector is compared against all stored chunk vectors, and the top-K most similar chunks are returned as candidate context.
HOW	– Dense retrieval: ANN search (cosine similarity or dot product) returns the K nearest vectors in sub-millisecond time. – Sparse retrieval (BM25/TF-IDF): keyword-overlap scoring complements dense search by catching exact-match terms the embedder may underweight. – Hybrid retrieval: combines dense and sparse scores via Reciprocal Rank Fusion (RRF) or learned linear interpolation. – Multi-query retrieval: the query is paraphrased N times, each paraphrase retrieves candidates, and results are merged and deduplicated.

WHY	– HyDE (Hypothetical Document Embeddings): the LLM generates a hypothetical answer first; that answer is embedded and used as the query vector.
	<i>Retrieval quality is the primary bottleneck in RAG. High recall at this stage ensures the right information reaches the LLM; low recall means the model will hallucinate regardless of how good the generation step is.</i>
	TIPS
	✓ Retrieve more candidates than you plan to use (e.g. top-20) and then rerank down to top-5. ✓ Apply metadata filters before ANN search to reduce the search space and improve precision. ✓ Log retrieval results and monitor mean reciprocal rank (MRR) or NDCG in production.

INSTALLATION REQUIREMENTS

Setting up your environment to build a RAG pipeline

1. Virtual Environment Setup

A virtual environment isolates your RAG project dependencies from the global Python installation, preventing version conflicts across projects.

Create & activate (macOS / Linux)

```
# Create a virtual environment named 'rag-env'
python3 -m venv rag-env

# Activate it
source rag-env/bin/activate

# Confirm the active interpreter
which python          # should point to rag-env/bin/python
```

Create & activate (Windows)

```
# Create a virtual environment
python -m venv rag-env

# Activate it (Command Prompt)
rag-env\Scripts\activate.bat

# Activate it (PowerShell)
rag-env\Scripts\Activate.ps1
```

Deactivate & manage

```
# Deactivate the environment when done
deactivate
```

```
# Save current dependencies to a file
pip freeze > requirements.txt

# Restore dependencies in a new environment
pip install -r requirements.txt
```

2. LangChain Installation

LangChain is the primary orchestration framework for RAG pipelines. It provides ready-made components for document loading, chunking, embedding, vector store integration, and chain composition.

Core install (required)

```
# Core LangChain framework
pip install langchain

# LangChain community integrations (loaders, vector stores, etc.)
pip install langchain-community

# OpenAI models + embeddings
pip install langchain-openai
```

Document parsing & chunking

```
# PDF parsing
pip install pypdf

# Advanced layout-aware parsing (tables, multi-column PDFs)
pip install unstructured[pdf]

# HTML parsing
pip install BeautifulSoup4 lxml
```

Vector store — Weaviate

```
# Weaviate Python client (v4 - current stable)
pip install weaviate-client

# LangChain integration for Weaviate
pip install langchain-weaviate

# Run Weaviate locally with Docker (recommended for development)
docker pull semitechnologies/weaviate:latest
docker run -d -p 8080:8080 -p 50051:50051 \
  -e AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED=true \
  -e PERSISTENCE_DATA_PATH=/var/lib/weaviate \
  semitechnologies/weaviate:latest

# Connect in Python
import weaviate
client = weaviate.connect_to_local()          # local Docker instance
# client = weaviate.connect_to_weaviate_cloud() # Weaviate Cloud (WCD)
```

```
# cluster_url="https://<your-cluster>.weaviate.network",
# auth_credentials=weaviate.auth.AuthApiKey("<WCD-API-KEY>")
# )
```

Reranking models

```
# sentence-transformers – includes BGE & Hugging Face cross-encoders
pip install sentence-transformers

# Hugging Face Hub client (browse & download any reranker model)
pip install huggingface-hub
huggingface-cli download BAAI/bge-reranker-v2-m3

# Usage: BGE reranker (local, open-source)
from sentence_transformers import CrossEncoder
model = CrossEncoder("BAAI/bge-reranker-large")
scores = model.predict([("query", "passage 1"), ("query", "passage 2")])

# Usage: Hugging Face ms-marco cross-encoder (general purpose)
model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
scores = model.predict([("query", "passage")])

# Popular Hugging Face reranker models:
#   BAAI/bge-reranker-large           – best accuracy, ~560 MB
#   BAAI/bge-reranker-v2-m3          – multilingual, compact
#   cross-encoder/ms-marco-MiniLM-L-6-v2 – fast, lightweight
#   cross-encoder/ms-marco-TinyBERT-L-2 – ultra-fast, smallest
```

Install everything at once

```
pip install langchain langchain-community langchain-openai \
    pypdf unstructured[pdf] beautifulsoup4 \
    weaviate-client langchain-weaviate \
    sentence-transformers huggingface-hub
```

Environment variables

```
# Create a .env file in your project root
OPENAI_API_KEY=sk-...
HUGGINGFACE_HUB_TOKEN=hf-...

# Load it in Python
pip install python-dotenv

from dotenv import load_dotenv
load_dotenv() # reads .env automatically
```

EVALUATION FRAMEWORK

Testing and measuring RAG pipeline quality with DeepEval

1. What is DeepEval?

DeepEval is an open-source LLM evaluation framework purpose-built for RAG and generative AI pipelines. It provides a pytest-like interface to unit-test LLM outputs against a suite of metrics — from retrieval faithfulness to answer relevance — without relying on a single proprietary judge.

Key capabilities

- ▶ RAG-specific metrics: Faithfulness, Answer Relevancy, Contextual Precision, Contextual Recall, Contextual Relevancy.
- ▶ LLM-as-a-judge: uses a configurable judge model (GPT-4o, local Ollama, etc.) to score free-text outputs.
- ▶ pytest integration: write eval test cases in standard Python test files; run with `deepeval test run`.
- ▶ Dataset management: create, version, and store evaluation datasets (called EvaluationDatasets) locally or on Confident AI.
- ▶ Regression testing: track metric scores across pipeline versions to catch quality regressions in CI/CD.
- ▶ Conversational & agentic evals: evaluate multi-turn conversations and tool-calling agents, not just single Q&A.

2. Installation

Install DeepEval

```
pip install deepeval

# Verify installation
deepeval --version

# Optional: login to Confident AI dashboard (cloud observability)
deepeval login
```

Configure the judge model

```
# DeepEval uses an LLM judge to score metrics — set your key
export OPENAI_API_KEY=sk-...

# Or use a local Ollama model as the judge (no API key needed)
from deepeval.models import OllamaModel
judge = OllamaModel(model='llama3')
```

3. Core RAG Metrics

Each metric takes a threshold (0–1). A test case passes if the score meets or exceeds the threshold. All metrics use an LLM judge internally unless otherwise noted.

Metric	What it measures	Input required
Faithfulness	Does the answer contain only claims supported by the retrieved context? Catches hallucinations.	input, actual_output, retrieval_context
Answer Relevancy	Is the answer directly relevant to the question? Penalises off-topic or verbose responses.	input, actual_output
Contextual Precision	Are the retrieved chunks ranked so the most relevant ones appear first?	input, expected_output, retrieval_context

Contextual Recall	Does the retrieved context cover all facts needed to produce the expected answer?	input, expected_output, retrieval_context
Contextual Relevancy	Are the retrieved chunks actually relevant to the question (regardless of rank)?	input, retrieval_context
Hallucination	Does the actual output contradict the provided context or introduce fabricated facts?	input, actual_output, context

Running metrics in code

```
from deepeval import evaluate
from deepeval.metrics import FaithfulnessMetric, AnswerRelevancyMetric
from deepeval.test_case import LLMTestCase

test_case = LLMTestCase(
    input='What is RAG?',
    actual_output='RAG stands for Retrieval-Augmented Generation...',
    retrieval_context=['RAG combines retrieval with generation...']
)

faithfulness = FaithfulnessMetric(threshold=0.7)
relevancy = AnswerRelevancyMetric(threshold=0.7)

evaluate([test_case], [faithfulness, relevancy])
```

4. Golden Dataset Generation

A golden dataset is a curated set of (question, expected_answer, ground_truth_context) triples used as the ground truth for evaluation. DeepEval can auto-generate golden datasets from your documents using an LLM, dramatically reducing the manual effort of building an eval set.

Why a golden dataset matters

- Provides a reproducible benchmark — run the same eval set after every pipeline change to detect regressions.
- Covers edge cases: generated goldens can include multi-hop questions, negations, and out-of-scope queries.
- Removes subjectivity: expected outputs act as objective targets for Contextual Recall and Precision metrics.
- Enables A/B testing between embedding models, chunk sizes, or reranker configurations.

Generate goldens from documents

```
from deepeval.synthesizer import Synthesizer
from deepeval.dataset import EvaluationDataset

synthesizer = Synthesizer() # uses OpenAI GPT-4o by default

# Generate question-answer pairs from your source documents
synthesizer.generate_goldens_from_docs(
    document_paths=['docs/handbook.pdf', 'docs/faq.txt'],
    include_expected_output=True, # generates reference answers
    max_goldens_per_document=25,
)
```

```
# Wrap in an EvaluationDataset and save
dataset = EvaluationDataset(goldens=synthesizer.synthetic_goldens)
dataset.save_as(file_type='json', directory='./eval_data')

# Inspect a sample golden
g = dataset.goldens[0]
print(g.input)           # generated question
print(g.expected_output) # reference answer
print(g.context)         # source chunk(s) used
```

Generate goldens from existing Q&A pairs

```
# If you already have questions, generate only the expected outputs
synthesizer.generate_goldens_from_scratch(
    subject='company HR policies',
    task='question answering',
    input_format='A specific HR policy question an employee might ask',
    num_initial_goldens=50,
    num_evolution=2,      # evolve simple questions into harder variants
)
```

Run full eval against the golden dataset

```
from deepeval import evaluate
from deepeval.metrics import (
    FaithfulnessMetric, AnswerRelevancyMetric,
    ContextualPrecisionMetric, ContextualRecallMetric
)

# Load saved dataset
dataset = EvaluationDataset()
dataset.pull(alias='rag-golden-v1') # from Confident AI, or load from JSON

# Run your RAG pipeline on each golden to produce actual outputs
test_cases = []
for golden in dataset.goldens:
    actual_output = my_rag_pipeline(golden.input) # your RAG function here
    test_cases.append(LLMTestCase( input=golden.input,
    actual_output=actual_output, expected_output=golden.expected_output,
    retrieval_context=golden.context, ))

metrics = [
    FaithfulnessMetric(threshold=0.7),
    AnswerRelevancyMetric(threshold=0.7),
    ContextualPrecisionMetric(threshold=0.6),
    ContextualRecallMetric(threshold=0.6),
]

# Now evaluate against the assembled test cases
dataset.evaluate(test_cases, metrics) # returns pass/fail per test case
```

Integrate with pytest

```
# test_rag.py
```

```

import pytest
from deepeval import assert_test
from deepeval.metrics import FaithfulnessMetric
from deepeval.test_case import LLMTestCase

test_cases = []
for golden in dataset.goldens:
    actual_output = my_rag_pipeline(golden.input) # your RAG function
    test_cases.append(LLMTestCase( input=golden.input,
    actual_output=actual_output, expected_output=golden.expected_output,
    retrieval_context=golden.context, ))

@pytest.mark.parametrize('test_case', dataset.test_cases)
def test_rag_faithfulness(test_case: LLMTestCase):
    assert_test(test_case, [FaithfulnessMetric(threshold=0.7)])

# Run all evals:
# deepeval test run test_rag.py

```